How to set up and run

Setup, run, test, and check the two new features

Requirements

- Node.js 22.14 or newer
- pnpm 10.18.3 (enable with `corepack enable`)
- Git, and access to the private repo Aman-0402/field-desk

Setup

```
git clone https://github.com/Aman-0402/field-desk.git
cd field-desk
pnpm install
pnpm env:setup      # creates the .env files
pnpm db:migrate     # applies all migrations, including 0006
pnpm admin:create   # asks for admin name, email, password
pnpm db:seed        # optional: sample/reference data (SKUs etc.)
```

Run the app

```
pnpm dev           # starts client and server together
```

Open the URL printed in the terminal by the client (Vite normally uses http://localhost:5173) and log in with the admin you created using pnpm admin:create. See docs/operations.md for details.

Run the checks

Command	What it does
pnpm lint	ESLint
pnpm typecheck	TypeScript for shared, client, server
pnpm test	Compiles and runs all 265 tests (about 2 minutes)
pnpm check	Lint, typecheck, test and build in one go
pnpm metrics	Writes reports/source-metrics.json (files, lines, decision nodes)
pnpm db:generate	Generate a new Drizzle migration after a schema change

Run only the new tests after a build: `cd server && npx tsc -p tsconfig.test.json && node --test dist-test/test/corrections.test.js`

Using the screens

- **Correct delivery** (KAM or Admin): open an order, go to the **Shipments** tab, click **Correct delivery**, enter the quantity that actually arrived, add a reason.
- **Move payment** (Accounts or Admin): open a customer, go to **Payments & credits, Payments**, click **Move** next to the payment, choose the correct customer, add a reason.
- Screenshots of both are in docs/demo/ in the repo.

How to check FDSK-170 (delivery correction)

1. Create a customer and an order for 10 units; ship all 10 and mark them delivered.
2. Record a payment that covers the full amount. The customer shows no outstanding balance.
3. As KAM, send `CORRECT_DELIVERY` with deliveredQty = 6.
4. Expected: the shipment is back to Dispatched; the charge is for 6 units; the extra cash appears as unapplied (free) cash; points for 4 units are locked again.
5. As Warehouse, confirm the remaining 4. Expected: free cash is applied, shipment becomes Delivered, points return.

How to check FDSK-169 (move a payment)

1. Create two customers, each with a delivered order.
2. Record a large payment against customer A by mistake. A shows nothing outstanding.
3. As Accounts, send `MOVE_PAYMENT` with the receipt id and toCustomerId = B.
4. Expected: A's charges reopen and A's points from that cash are gone; B's charges are settled and B earns the points; A's old receipt is still visible, marked reversed, and B's new receipt points back to it.

Sending a command by API

Both commands go through `POST /api/actions` inside an envelope (see docs/api.md). The shapes are:

```
MOVE_PAYMENT     { customerId, receiptId, toCustomerId, reason }
CORRECT_DELIVERY { orderId, shipmentId,
                   items: [{ shipmentLineId, deliveredQty }], reason }
```

The easiest working example is server/test/corrections.test.ts, which builds the envelopes through the HttpDesk test helper. Both commands also have screens in the app (see the steps above).

Troubleshooting

- **Migration errors:** delete the local SQLite file and run pnpm db:migrate, pnpm admin:create and pnpm db:seed again.
- **Test timeouts:** the suite runs files one at a time on purpose; do not raise concurrency.
- **Wrong Node version:** check with node -v; it must be 22.14 or newer.